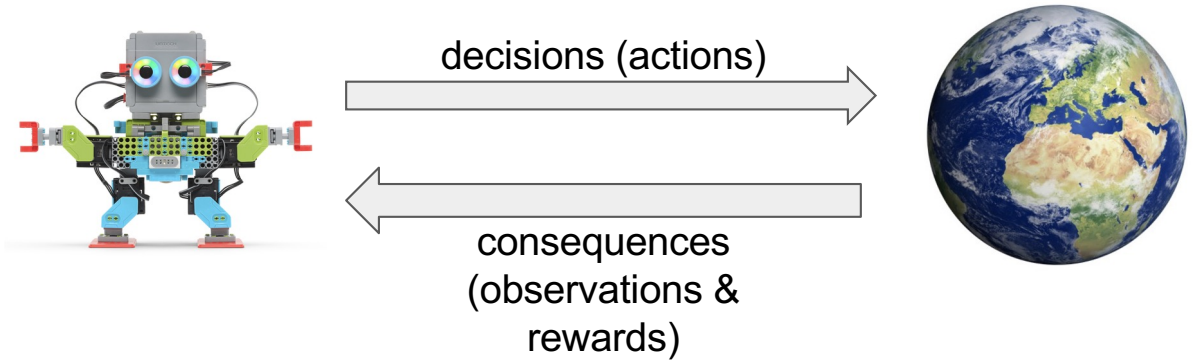
# INTRODUCTION TO RL

# REINFORCEMENT LEARNING

# REINFORCEMENT LEARNING



# REINFORCEMENT LEARNING

- RL allows the machine or software agent to learn its ideal behavior based on feedback from the environment i.e. REWARDS. Below is a block diagram of a typical RL setup.

The underlying principle behind RL :-

- Law of Effect :-
  - You do something, something good happens, so you try to do it more often if a similar situation arises.
  - You do something, something bad happens, so you try to do it less often if a similar situation arises.

# WHY SEQUENTIAL DECISION MAKING ?

- Limited Supervision : you know **what** you want but not **how** to get it
- Actions have consequences
- Unlike classification, regression : when your system is making single isolated decision; when that decision doesn’t affect future decisions

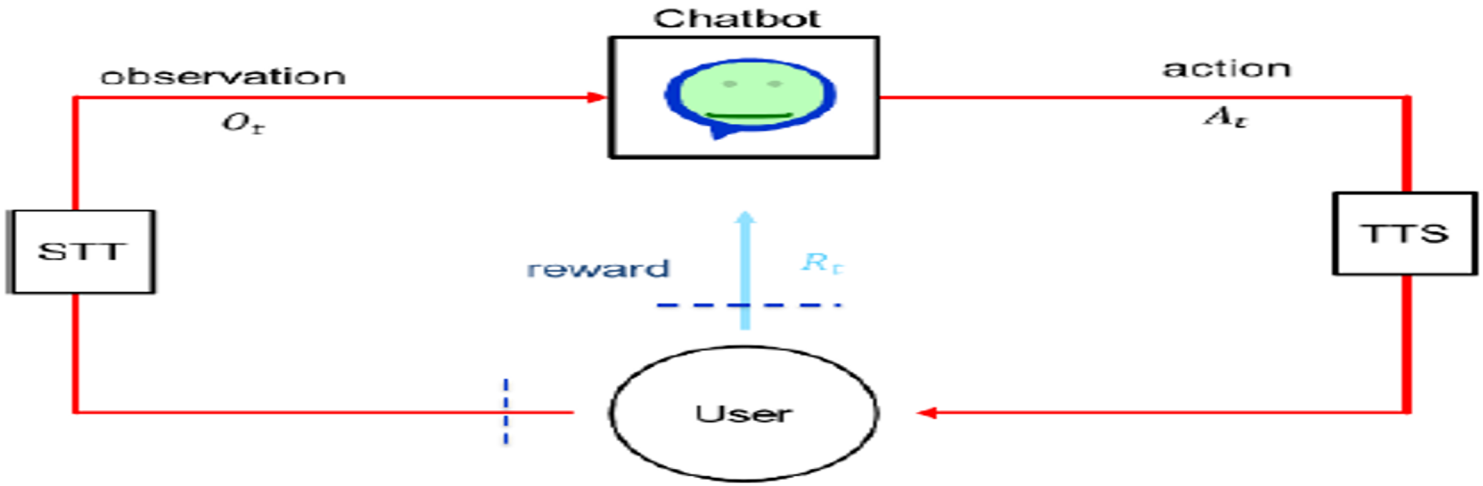
# REINFORCEMENT LEARNING

- Thus, any RL setup is mathematically described in terms of an abstraction called the Markov Decision Process (MDP)
- A MDP model is described by M = < S; A; R; P; Rho >, where
  - S is the finite state space
  - A is the finite action space (corresponding to VA's action)
  - R is the reward model
  - P is the transition model
  - Rho is the initial state distribution

# A RL CHATBOT

- The RL Chatbot takes an action ($a_t$) at time t which is converted into speech by text-to-speech (TTS) unit
- The user's response, in the form of speech is converted to text by speech-to-text (STT) unit
- The output of the STT unit i.e. the observation ($o_t$) is received by the chatbot
- The user may either generate a reward signal ($r_t$) explicitly or it is derived from the user's response
- The task of the RL Chatbot is to respond in a way so as to maximize it's reward

# A RL CHATBOT



# WHY DEEP RL ?

Deep Learning : end to end training of expressive multi-layer models

These deep models allows RL algorithms to solve complex problems end to end

# CHARACTERISTICS OF RL ?

- Exploration - Exploitation trade off
- Delayed Rewards

# Q-LEARNING

- Q-learning defines a function *Q(s,a)* which represents the **discounted future reward** when we perform action *a* in state *s*, and continue optimally from that point on.
- The way to think about *Q(s,a)* is that it is "the best possible score at the end of an episode after performing action *a* in state *s*"
- It is called Q-function, because it represents the "quality" of certain action in given state.
- If you want to select the action that results in the highest score at the end of an episode
  - Pick the action with the highest Q-value : $\pi(s)=argmax_a Q(s,a)$
- The main idea in Q-learning is that we can iteratively approximate the Q-function using the Bellman equation
  - $Q(s,a) = r + \gamma\ max_{a'} Q(s',a')$
  - Here $\gamma$ is the discount factor between 0 and 1 – the more into the future the reward is, the less we take it into consideration